BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên	Lê Minh Hiếu
MSSV / Lớp	3124720023

Quy ước: goal test khi lấy node ra khỏi frontier; "Explored Set" ghi theo thứ tự node được lấy ra; hòa thì phá theo thứ tự alphabet.

Bài 1. Các thuật toán tìm kiếm

Các cạnh: S→A (3), S→C (2), S→D (2), C→F (1), D→B (3), D→G (8), B→E (2), F→E (0), F→G (4), E→G (2). Nút A không có cạnh đi ra.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, F, B, G
Path	S → D → G
Total Cost	2 + 8 = 10

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E, G
Path	S → C → F → E → G
Total Cost	2 + 1 + 0 + 2 = 5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G (tìm thấy G)

Path cuối cùng: S → D → G

Total Cost: 10

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

- **Frontier:** BFS dùng hàng đợi FIFO nên duyệt theo từng tầng. DFS dùng ngăn xếp LIFO nên đi sâu hết nhánh rồi mới quay lui.
- **Tính tối ưu:** BFS tìm được đường đi ít bước nhất khi chi phí mỗi bước như nhau, nhưng không đảm bảo đường có tổng chi phí nhỏ nhất khi các cạnh có

trọng số khác nhau. Trong bài này BFS cho đường đi có cost 10, còn DFS tìm được đường có cost 5. DFS cũng không đảm bảo tìm được đường tối ưu và có thể gặp vấn đề nếu đồ thị có chu trình.

Bài 2. Tháp Hà Nội – Tower of Hanoi

2.1. Biểu diễn trạng thái

Một trạng thái là bộ n phần tử $s = (p_1, p_2, \ldots, p_n)$, với $p_i \in \{1, 2, 3\}$ là cọc chứa đĩa i (đĩa 1 nhỏ nhất, đĩa n lớn nhất). Thứ tự đĩa trên mỗi cọc luôn được xác định (lớn dưới, nhỏ trên), nên biểu diễn này đủ để xác định các hành động hợp lệ.

2.2. Kích thước không gian trạng thái

Mỗi đĩa độc lập chọn 1 trong 3 cọc, và mọi cách xếp đều hợp lệ vì trên mỗi cọc các đĩa luôn tự sắp theo kích thước. Vậy có $\mathbf{3^n}$ trạng thái.

2.3. Trạng thái bắt đầu

$S_0 = (1, 1, \ldots, 1)$, tức tất cả đĩa ở cọc 1.

2.4. Các hành động hợp lệ

Actions(s) = {move(i, a→b)}, với $a = p_i$ và $b \neq a$, thỏa mãn:

1. Đĩa i đang ở cọc a và là đĩa trên cùng của cọc đó, tức không có đĩa $j < i$ nào có $p_j = a$.
2. Đĩa i chỉ được đặt lên cọc b nếu cọc b đang trống hoặc đĩa nằm trên cùng của cọc b lớn hơn đĩa i.
3. Mỗi lần chỉ chuyển một đĩa.

Mỗi trạng thái có tối đa 3 hành động hợp lệ.

2.5. Goal Test

goalTest(s) trả về True khi và chỉ khi $p_i = 3$ với mọi i = 1..n, tức s = (3, 3, …, 3). Số bước tối thiểu là $2^n - 1$.

Bài 3. Thiết kế và phân tích Heuristic

Các cạnh: S→A (5), S→B (3), A→C (4), A→D (9), B→D (1), C→D (2), C→G (6), D→G (4).

Chi phí thật h*: G = 0, D = 4, C = 6, B = 5, A = 10, S = 8.

3.1. Thiết kế heuristic admissible và consistent

Node	h(n)
S	7
A	9
B	4
C	5

Node	h(n)
D	3
G	0

- **Admissible:** mọi $h(n) \le h^*(n)$.
- **Consistent:** S→A: $7 \le 5+9$. S→B: $7 \le 3+4$. A→C: $9 \le 4+5$. A→D: $9 \le 9+3$. B→D: $4 \le 1+3$. C→D: $5 \le 2+3$. C→G: $5 \le 6+0$. D→G: $3 \le 4+0$. Tất cả đều thỏa.

3.2. Chạy A* với heuristic đã thiết kế

- Mở S: A có f = 5+9 = 14, B có f = 3+4 = 7.
- Mở B: D có g = 4, f = 7.
- Mở D: G có g = 8, f = 8.
- Lấy G (f = 8) là đích.

Explored Set: S, B, D, G

Path: S → B → D → G

Total Cost: 8

3.3. Heuristic admissible nhưng không consistent

h(S) = 8, h(A) = 10, h(B) = 5, h(C) = 6, h(D) = 1, h(G) = 0.

- Admissible vì mọi $h \le h^*$ (D: $1 \le 4$).
- Cạnh vi phạm: C→D có $h(C) = 6 > c(C,D) + h(D) = 2 + 1 = 3$. Cạnh B→D cũng vi phạm: $5 > 1 + 1 = 2$.

3.4. Câu hỏi mở rộng: consistent ⇒ admissible

Lấy đường đi tối ưu $n = n_0 \to n_1 \to \dots \to n_k = G$. Áp dụng consistency liên tiếp:

$h(n_0) \le c(n_0,n_1) + h(n_1) \le c(n_0,n_1) + c(n_1,n_2) + h(n_2) \le \dots \le \Sigma c + h(G) = \Sigma c + 0 = h^*(n)$.

Vậy $h(n) \le h^*(n)$, tức admissible.

Bài 4. Treasure Hunting (Tìm kho báu)

Các cạnh: A→C (2), A→D (1), A→E (3), B→A (2), C→F (2), C→G (1), D→G (1), D→H (3), E→H (1), E→I (1), F→B (3). Kho báu ở E và G (nút đỏ). Heuristic: A = 2, B = 4, C = 1, D = 1, E = 0, F = 6, G = 0, H = 0, I = 0.

Thuật toán	Explored Set	Path returned	Cost
4.1 DFS	A, C, F, B, G	A → C → G	3
4.2 BFS	A, C, D, E	A → E	3
4.3 UCS	A, D, C, G	A → D → G	2
4.4 Greedy	A, E	A → E	3

Thuật toán	Explored Set	Path returned	Cost
4.5 A*	A, D, G	A → D → G	2

Ghi chú:

- **UCS:** sau khi mở A và D, frontier có C (2) và G (2) hòa nhau, nên chọn C theo alphabet. Mở C chỉ cho G với g = 3, kém hơn g = 2 đã có. Sau đó lấy G.
- *A: f(D)=2, f(C)=3, f(E)=3 nên chọn D. Sau đó G có f=2 và được chọn làm đích.*

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu lớn nhất	A→C→G	3
BFS	Độ sâu nhỏ nhất	A→E	3
UCS	g(n)	A→D→G	2
Greedy	h(n)	A→E	3
A*	g(n)+h(n)	A→D→G	2

Chỉ UCS và A* cho đường tối ưu (cost 2).

Bài 5. Câu hỏi Đúng/Sai

Câu 1 trong đề bị lỗi chữ ("ít nhất nhiều"), được hiểu là "DFS luôn mở rộng không nhiều hơn A*".

Câu	Đúng / Sai	Giải thích
Câu 1	Sai	DFS có thể đi vào nhánh sâu sai trước, còn A* với heuristic tốt đi thẳng tới đích. Ví dụ ở Bài 4, DFS mở 5 node còn A* chỉ mở 3. Nếu hiểu ngược lại ("DFS mở ít nhất bằng A*") thì cũng sai, vì DFS có thể may mắn đi thẳng vào đích ngay nhánh đầu.
Câu 2	Sai	Xe đi được nhiều ô trong một nước. Nếu A và B cùng hàng cách nhau 5 ô thì chỉ cần 1 nước đi, nhưng Manhattan = 5 > 1, nên heuristic overestimate.
Câu 3	Đúng	Pacman đi theo 4 hướng với chi phí mỗi bước là 1, còn khoảng cách Euclid là đường ngắn nhất giữa hai điểm, nên không bao giờ lớn hơn chi phí thật.
Câu 4	Sai	Tổng có thể vượt chi phí thật. Ví dụ $h_1 = h_2 = h^*$, thì $h_1 + h_2 = 2h^* > h^*$.
Câu 5	Sai	Consistent ⇒ admissible chứ không ngược lại. Ví dụ ở 3.3, heuristic admissible nhưng vi phạm consistency ở cạnh C→D.

Câu	Đúng / Sai	Giải thích
Câu 6	Sai	*Với A* graph search sử dụng closed/explored set và không mở lại node đã đóng, heuristic admissible thôi chưa đủ để đảm bảo tối ưu; heuristic consistent sẽ đảm bảo điều này *Ví dụ: S→A (1), S→B (1), A→C (2), B→C (1), C→G (3), với h(S)=0, h(A)=0, h(B)=4, h(C)=0, h(G)=0. Heuristic này admissible nhưng không consistent vì h(B)=4 > 1+h(C)=1. A có thể đóng C với g=3 trước khi mở B. Sau đó đường tốt hơn qua B có g=2 không được xét lại, dẫn đến đường S→A→C→G có cost 6 thay vì đường tối ưu S→B→C→G có cost 5.**

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)
Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n).